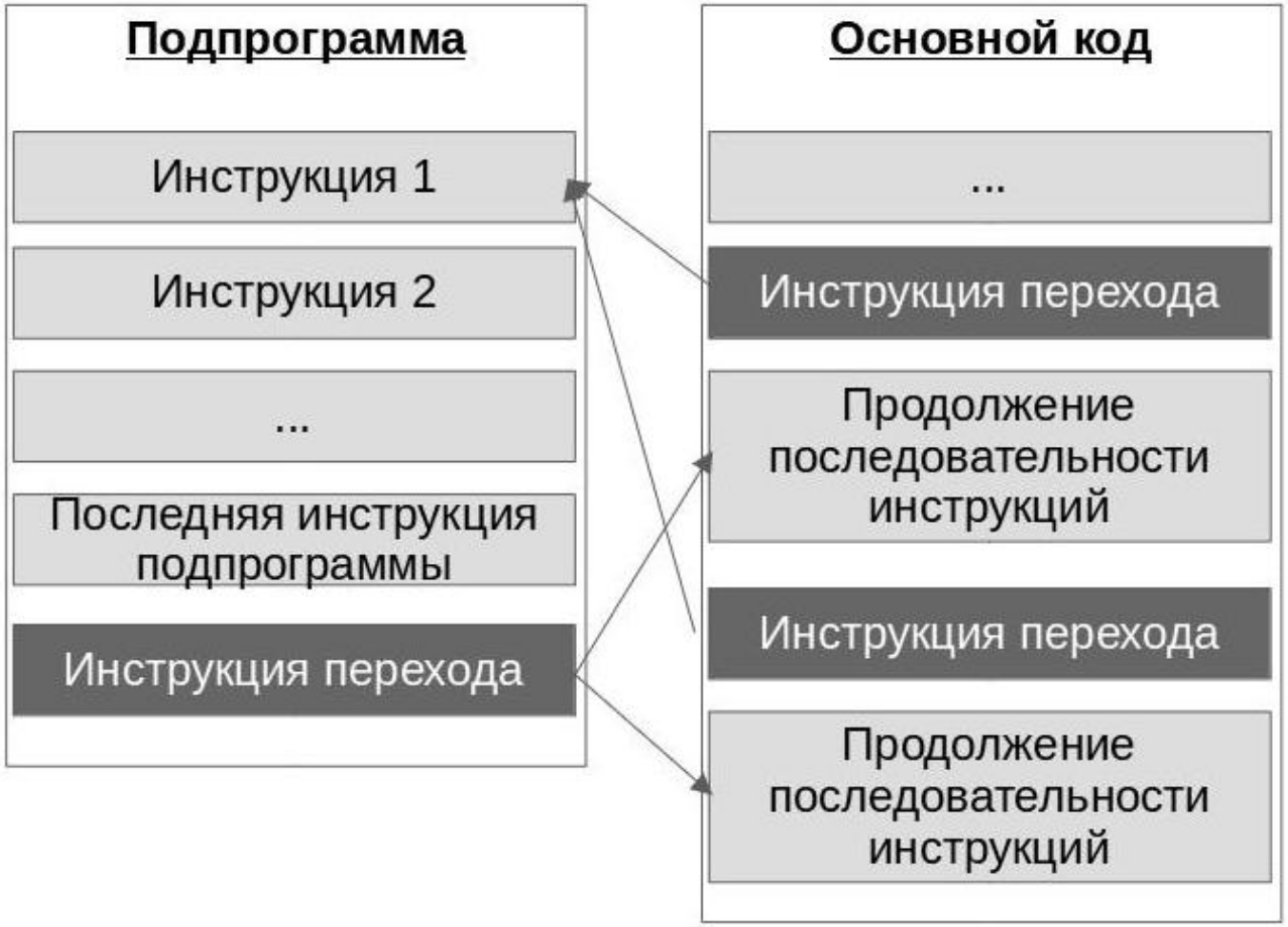
Модульное программирование

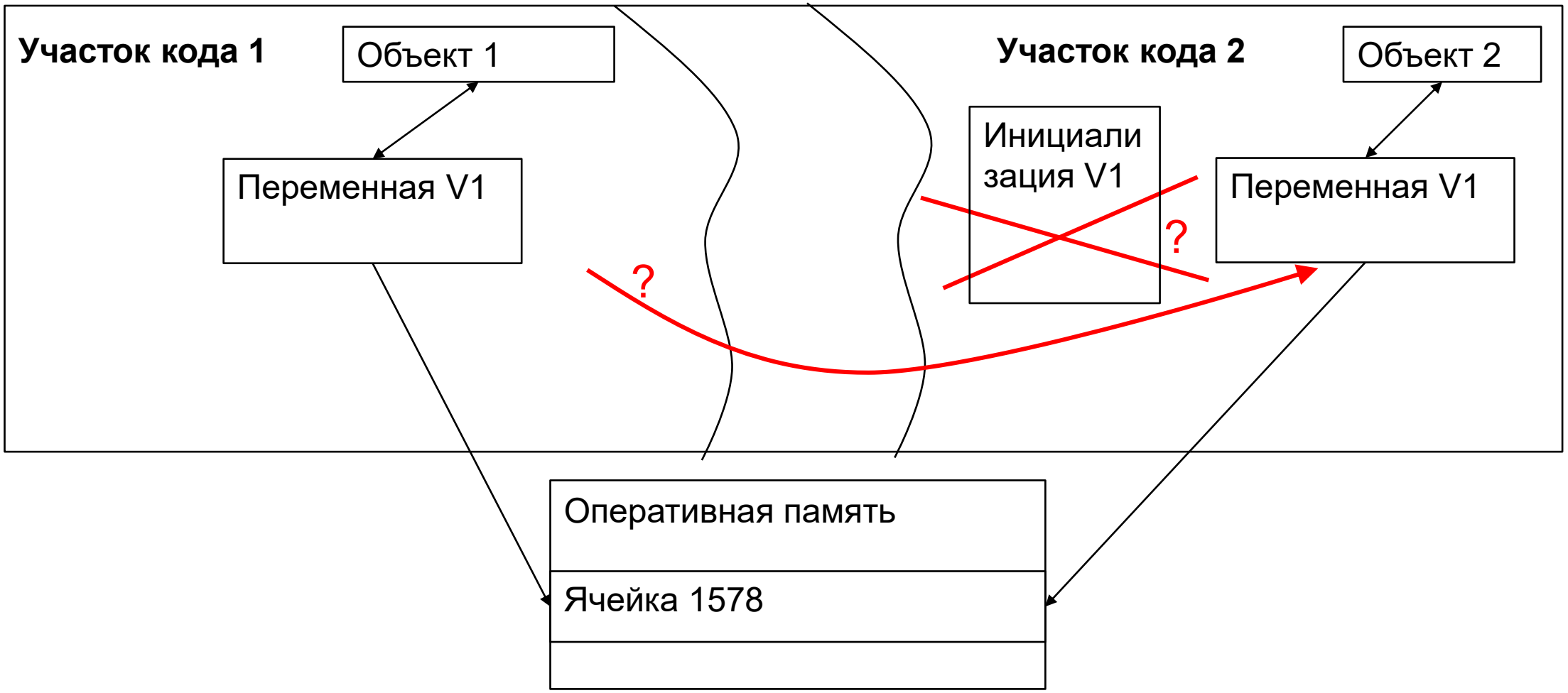
Курс «Алгоритмизация и программирование»,
лекция 4

Процедуры и функции

Имя подпрограммы – символьное обозначение последовательности операторов.



Проблема дублирования имён переменных



Определение подпрограмм

Определение подпрограммы – текст программы, расположенный отдельно от основного кода.

Вложенные подпрограммы – определение одной подпрограммы внутри определения другой подпрограммы.

Формальные параметры – переменные, начальные значения которых задаются в момент вызова подпрограммы.

Части определения подпрограммы:

- *заголовок подпрограммы;*
- *тело подпрограммы;*
- *оператор возврата управления.*

В заголовке подпрограммы задаются типы формальных параметров и тип возвращаемого значения.

Вызов подпрограммы

При вызове подпрограммы происходит:

- сохранение адреса возврата
- инициализация формальных параметров. Значения, используемые для инициализации называются фактическими параметрами
- передача управления первой инструкции подпрограммы

Область памяти программы

Область памяти, выделенная прикладной программе делится на следующие части:

- *сегмент кода.* В нём располагается машинный код программы, включая подпрограммы. Сегмент кода, как правило, доступен только для чтения;
- сегмент статических данных. В нём располагаются значения тех переменных, которые размещаются в оперативной памяти до начала выполнения программы;
- куча. В куче хранятся динамические переменные. Они размещаются в оперативной памяти в процессе выполнения программы.
- стек вызовов. Стек вызовов предназначен для хранения фактических параметров, передаваемых в подпрограмму, значений регистров процессора и адреса возврата. Стек вызовов организован по принципу «последний пришёл, первый ушёл» (LIFO).

При вызове подпрограммы в стеке вызовов формируется **стековый кадр**.

Стековый кадр

Стековый кадр используется для хранения:

- значений локальных переменных подпрограммы во время её выполнения,
- значений фактических параметров,
- адреса возврата,
- служебной информации, которая необходима подпрограмме, чтобы восстановить работу основного кода при возврате ему управления.

Для работы со стековым кадром используются регистры процессора:

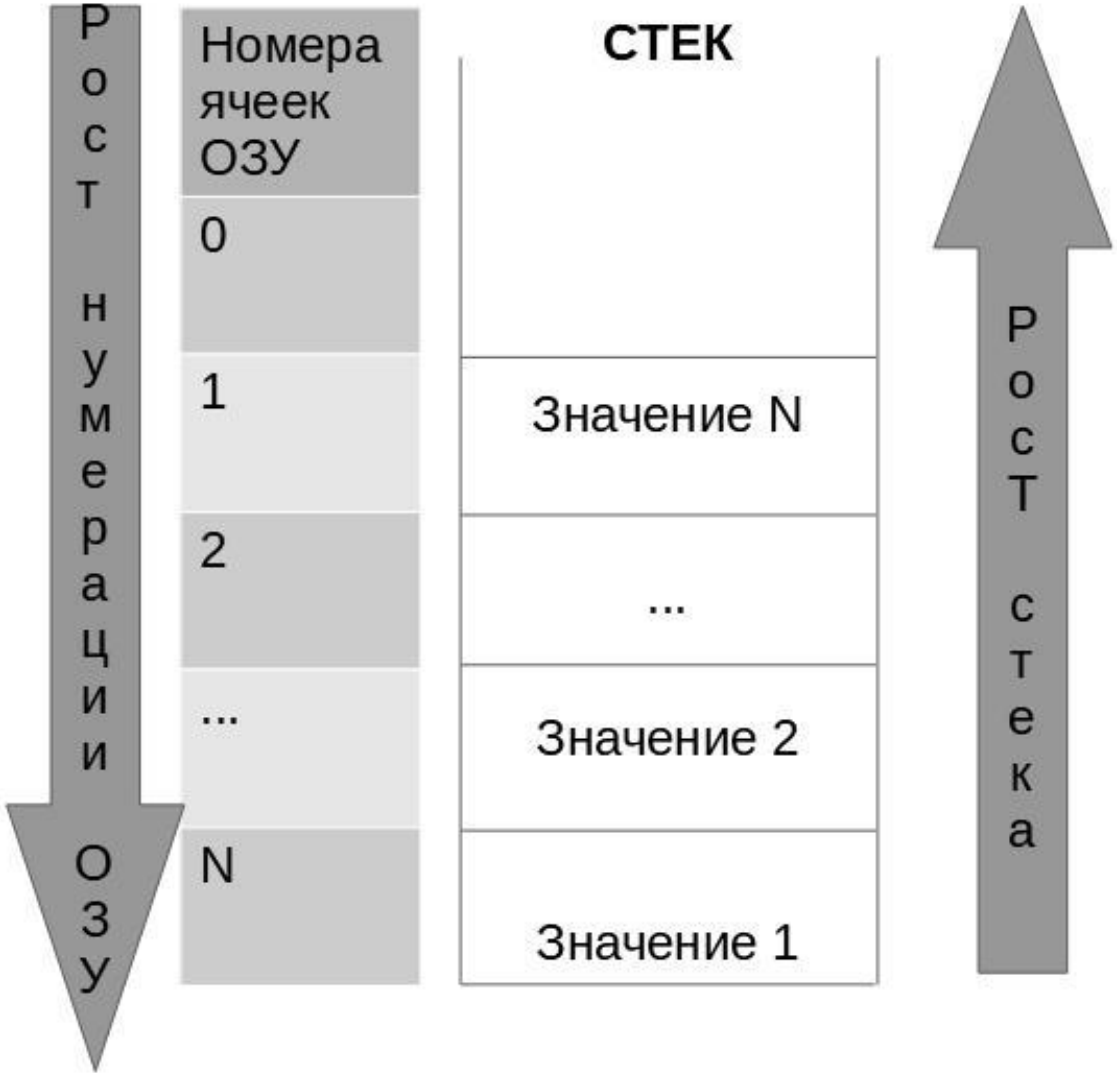
esp - указывает на вершину стека

ebp - указатель базы стекового кадра

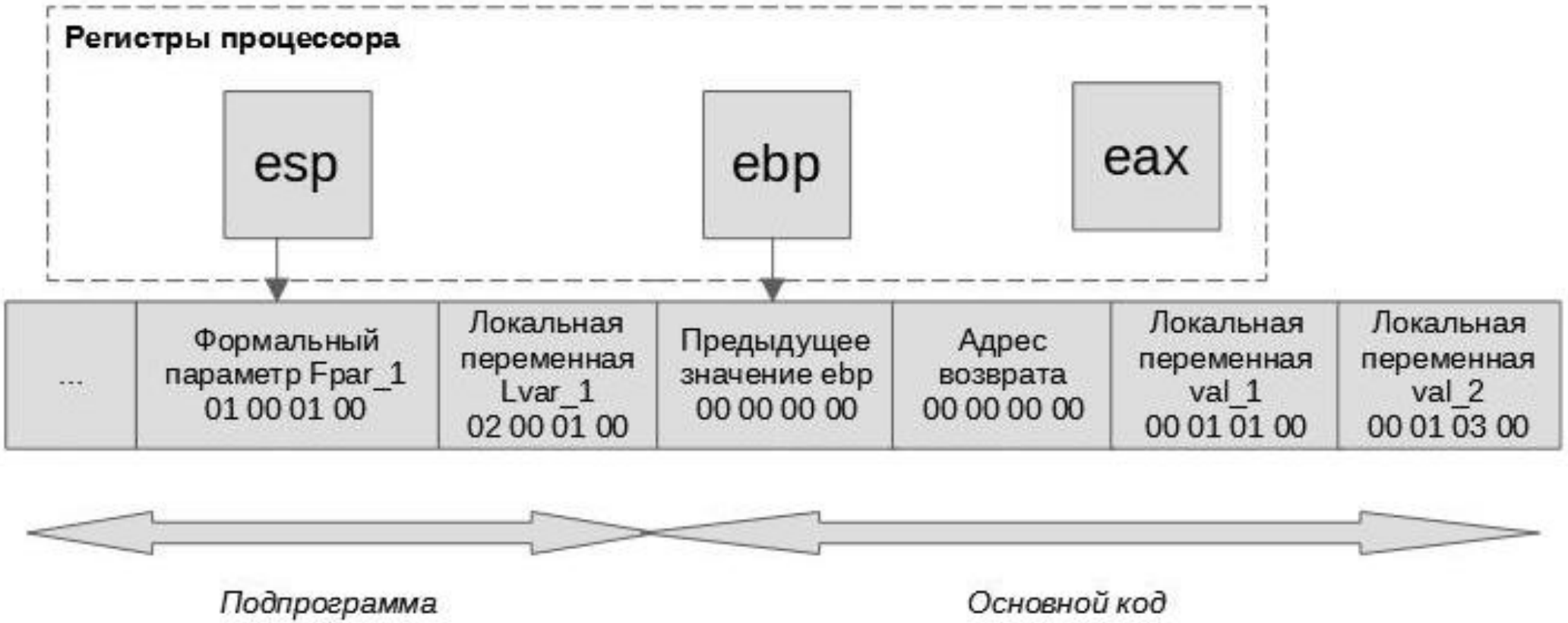
eax - регистр возврата. Через него в основной код передаётся значение, являющееся результатом работы подпрограммы.

Пролог – формирование стекового кадра и передача управления подпрограмме.

Направление роста стека возврата



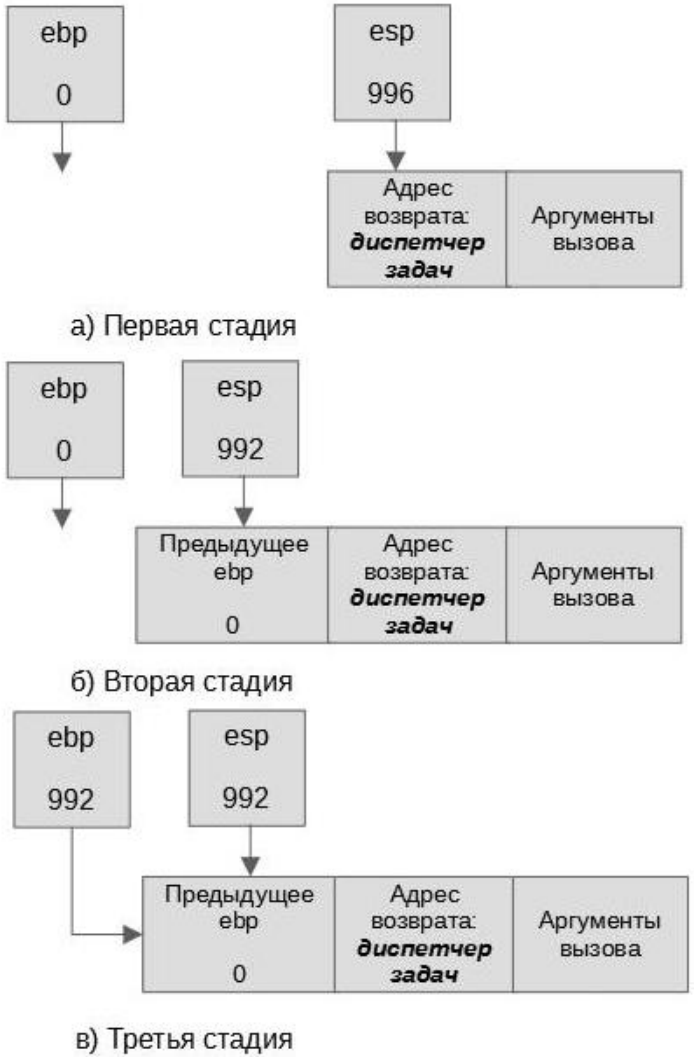
Начало формирования стекового кадра



Формирование стекового кадра, пример

```
ПОДПРОГРАММА sum_ab:
    формальные аргументы: a, b;
    возвращает значение целого типа
    { res = a + b
      завершить подпрограмму и вернуть значение res}
ОСНОВНОЙ КОД:
    x = 30
    y = 59
    goal = вызов полпрограммы sum_ab: фактические параметры x, y
    завершить ОСНОВНОЙ КОД
```

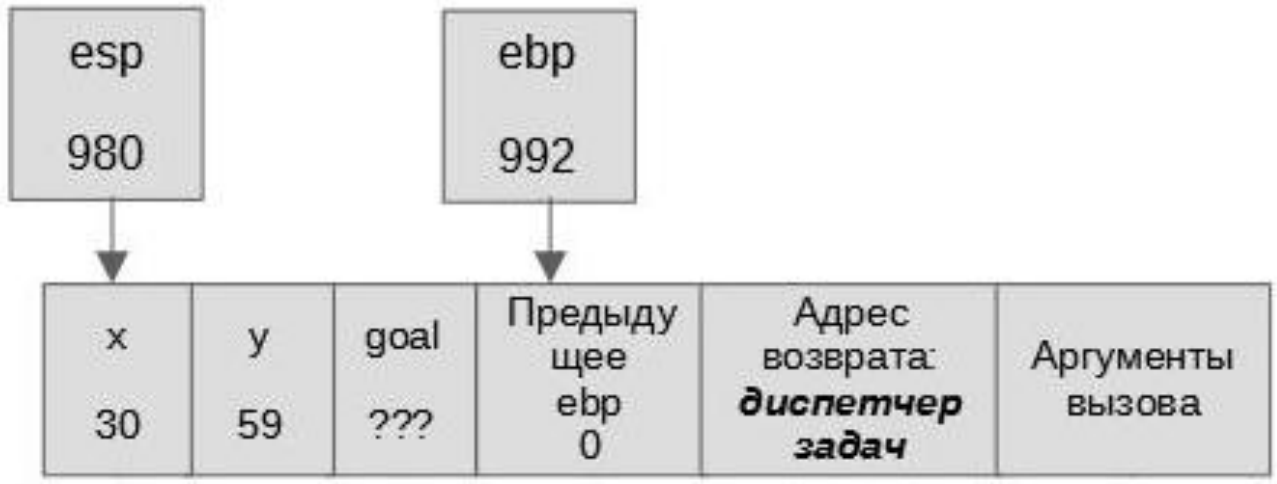
ЗАГРУЗКА ПРИКЛАДНОЙ ПРОГРАММЫ



Формирование стекового кадра, пример

```
ПОДПРОГРАММА sum_ab:
    формальные аргументы: a, b;
    возвращает значение целого типа
    { res = a + b
      завершить подпрограмму и вернуть значение res}
ОСНОВНОЙ КОД:
    x = 30
    y = 59
    goal = вызов полпрограммы sum_ab: фактические параметры x, y
    завершить ОСНОВНОЙ КОД
```

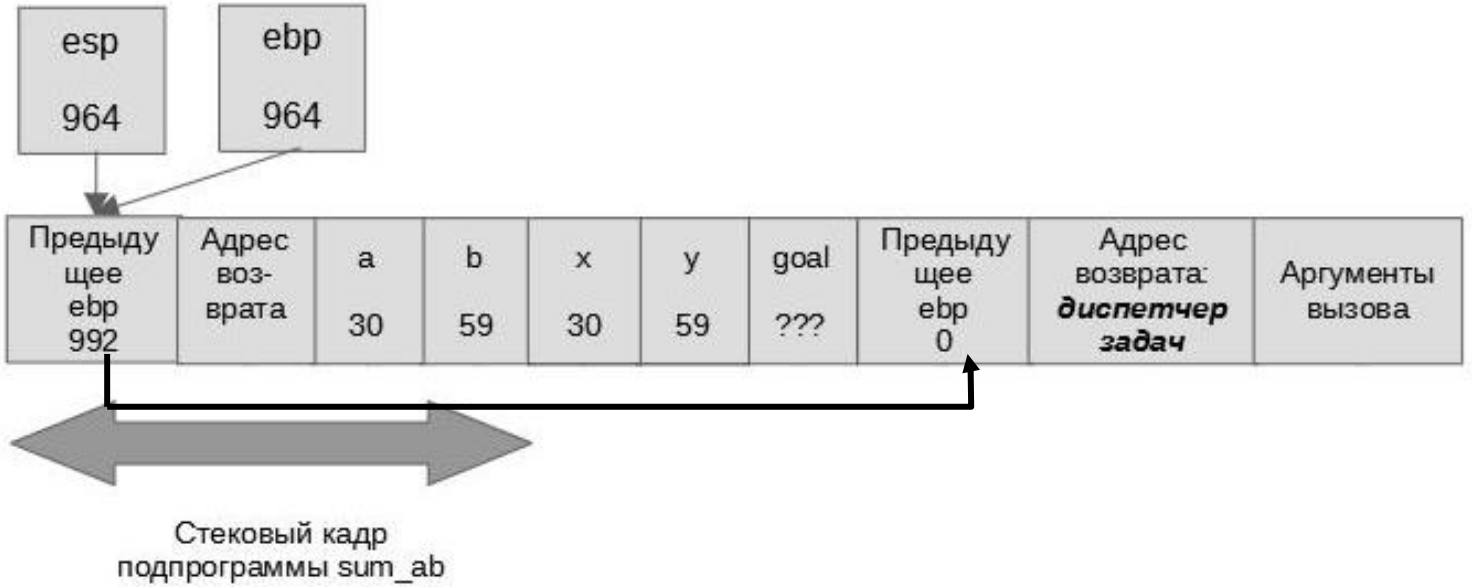
РАЗМЕЩЕНИЕ ЛОКАЛЬНЫХ ПЕРЕМЕННЫХ



Формирование стекового кадра, пример

```
ПОДПРОГРАММА sum_ab:
    формальные аргументы: a, b;
    возвращает значение целого типа
    { res = a + b
      завершить подпрограмму и вернуть значение res}
ОСНОВНОЙ КОД:
    x = 30
    y = 59
    goal = вызов полпрограммы sum_ab: фактические параметры x, y
    завершить ОСНОВНОЙ КОД
```

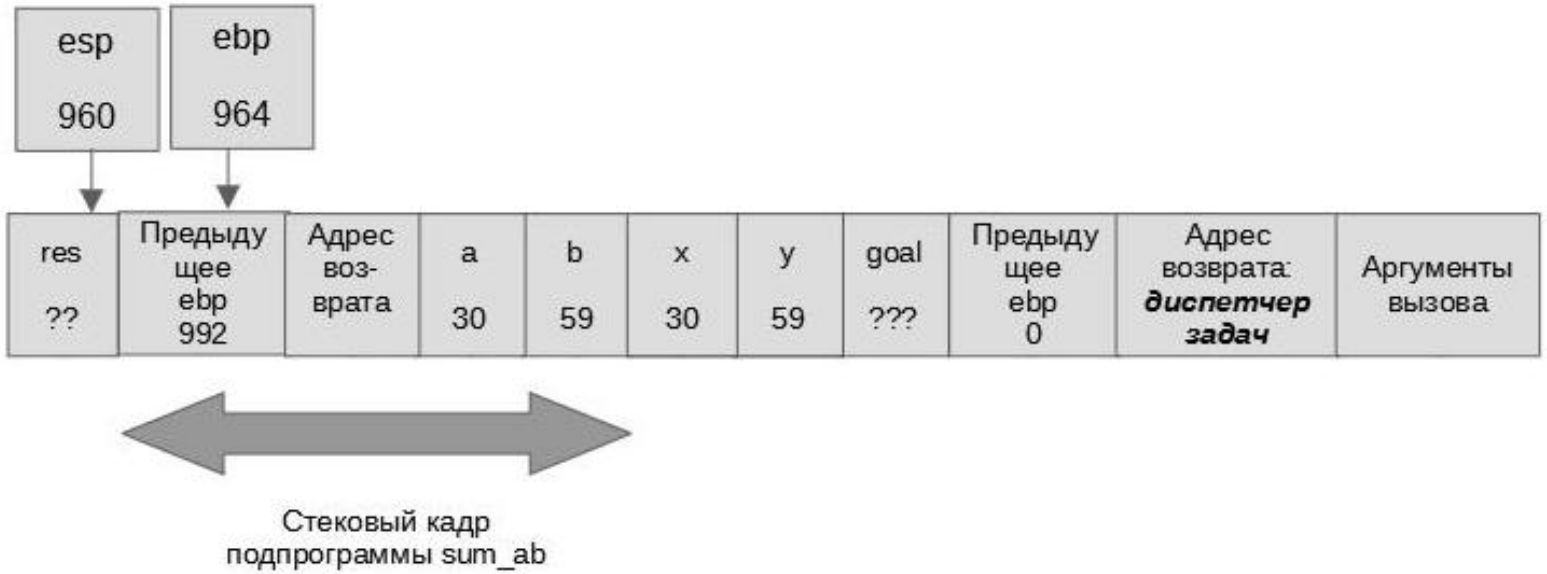
РАЗМЕЩЕНИЕ фактических параметров и адреса возврата



Формирование стекового кадра, пример

```
ПОДПРОГРАММА sum_ab:
    формальные аргументы: a, b;
    возвращает значение целого типа
    { res = a + b`````
      завершить подпрограмму и вернуть значение res}
ОСНОВНОЙ КОД:
    x = 30
    y = 59
    goal = вызов полпрограммы sum_ab: фактические параметры x, y
    завершить ОСНОВНОЙ КОД
```

РАЗМЕЩЕНИЕ локальной переменной

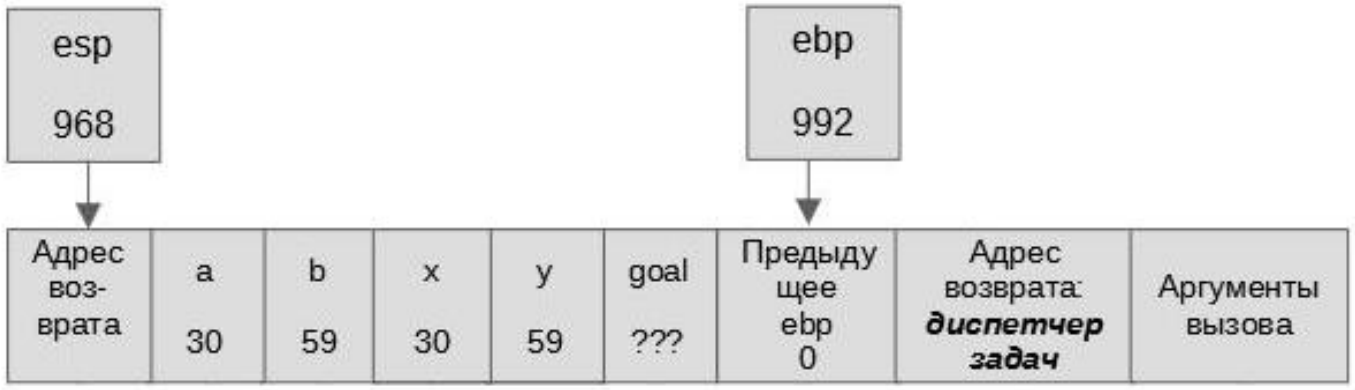


Эпилог, пример

Эпилог – завершение подпрограммы

```
ПОДПРОГРАММА sum_ab:
    формальные аргументы: a, b;
    возвращает значение целого типа
    { res = a + b`````
      завершить подпрограмму и вернуть значение res}
ОСНОВНОЙ КОД:
    x = 30
    y = 59
    goal = вызов полпрограммы sum_ab: фактические параметры x, y
    завершить ОСНОВНОЙ КОД
```

Удаление из стека локальных переменных и возврат указателя базы стекового кадра



Эпилог, пример

Корректировка esp: esp + <объём фактических параметров> + 4

```
ПОДПРОГРАММА sum_ab:
    формальные аргументы: a, b;
    возвращает значение целого типа
    { res = a + b`````
      завершить подпрограмму и вернуть значение res}
ОСНОВНОЙ КОД:
    x = 30
    y = 59
    goal = вызов полпрограммы sum_ab: фактические параметры x, y
    завершить ОСНОВНОЙ КОД
```

Корректировка esp



Возврат значения; процедуры и функции

Полная фраза:
«сохранение значения локальной переменной в регистре процессора при возврате управления из подпрограммы»
Сокращённая фраза:
«возврат значения».

Процедуры не возвращают значения.
Функции возвращают значение.

Значения целых чисел возвращаются через регистр eax процессора.
Значения чисел с плавающей точкой возвращаются через регистры R0:R1 сопроцессора.

ПРИМЕР
Вычислить значение: $z = t + y + f1(x)$.

Операция	Место хранения результатта
t + y	eax
сохранить eax	регистр “X” процессора
f1(x)	eax
регистр “X” в ecx	ecx
eax + ecx	ecx
копирование ecx в z	z

Возврат значения из процедуры

Сценарий 1. Вызывающая программа создаёт набор данных в куче. Адрес набора данных передаётся в подпрограмму как фактический параметр.

Сценарий 2. Вызываемая программа создаёт в куче набор данных. Адрес набора данных возвращается в вызывающую подпрограмму:

a. через регистр процессора eax. В этом случае процедура имеет признаки функции.
b. в куче создаётся переменная типа ССЫЛКА и её адрес передаётся в подпрограмму в числе фактических параметров. Подпрограмма сохраняет по этому адресу адрес созданного объекта. Используется адрес адреса.

Возможна комбинация функции и процедуры.

Передача простых переменных называется передача по значению.
Передача агрегатных переменных называется передача по адресу.

Недостаток подпрограмм

Использование подпрограмм увеличивает время выполнения программы и объём памяти, используемый программой.

Необходимо соблюдать баланс между следующими факторами:

- лёгкость понимания закодированного алгоритма;
- лёгкость тестирования;
- удобство документирования;
- увеличение времени выполнения машинного кода;
- увеличение объёма памяти, используемого программой.

Область видимости переменной

Область видимости переменной – часть кода алгоритма, в пределах которой имя переменной однозначно определяет ячейку оперативной памяти, в которой хранится значение.

Типы областей видимости:

- **Глобальная область видимости** – переменная видна во всём тексте кода. Область видимости начинается либо в начале текста, либо в точке определения переменной.
- **Локальная область видимости** – переменная видна в пределах подпрограммы, в которой она определена. Начало области видимости либо начало текста подпрограммы, либо точка, в которой переменная определена.
- **Видимость в пределах модуля.** Переменная, являющаяся глобальной в пределах модуля, имеет область видимости типа модуль.
- **Пространство имён.** Специальная инструментальная область видимости, объединяющая несколько областей видимости типа модуль.

Область видимости переменной

Способы задания области видимости

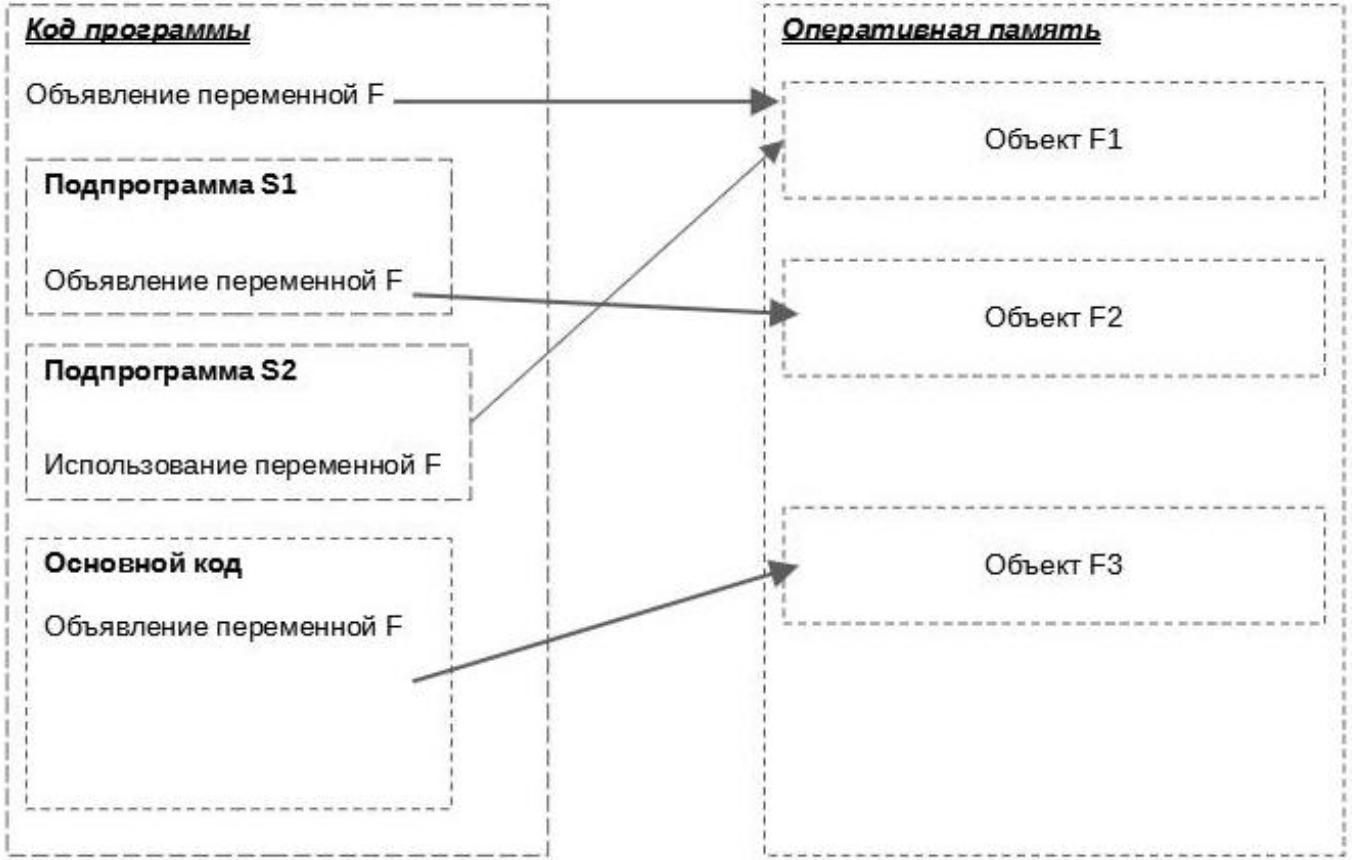
- Переменная, объявленная вне основной программы, подпрограмм или объявлений агрегатных типов данных, является ***глобальной для файла, в котором объявлена.***
- Переменная, объявленная в пределах кода подпрограммы, является ***локальной***. Переменная, объявленная внутри составного оператора, является ***локальной*** для этого оператора.
- Переменная, объявленная при описании агрегатного типа данных, будет иметь ту же область видимости, что и агрегатный тип данных.
- Переменные с областью видимости типа ***модуль*** видны в пределах всех файлов проекта.
- Видимость ***за пределами файла***:
  1) назначить переменную экспортируемой в том модуле, в котором она объявлена.
  2) указать, что переменная является импортируемой, в том модуле, в котором она используется.

Примечание: Расширением инструмента экспорт/импорт является инструмент создания пространства имён.

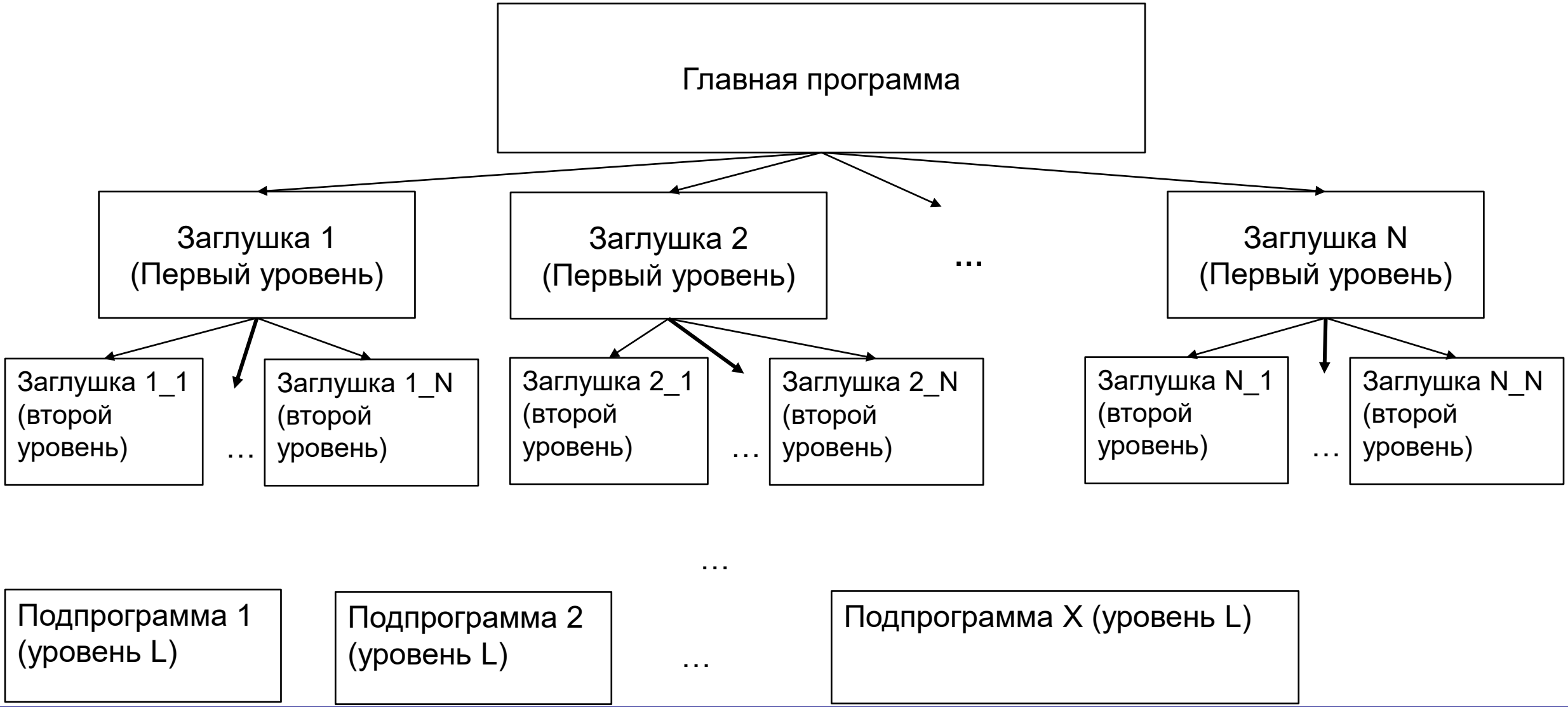
Область видимости переменной, коллизии

Коллизия – различные объекты обозначаются одинаковым именем.

Иерархия областей видимости:
глобальные → пространство имён → модуль → локальные → блок (составной оператор).



Кризис программирования; структурное программирование



Безусловный переход под запретом

Отрицательное влияние оператора GoTo:

1) Переменная объявляется в точке A программного кода, инициализируется в точке B программного кода и используется в точке D программного кода. Оператор GoTo, находящийся после точки A, позволяет передать управление в точку D, минуя точку B
2) Оператор GoTo позволяет передать управление телу цикла минуя заголовок цикла. В результате не произойдёт инициализация переменной цикла и/или первоначальной проверки условия окончания цикла.
3) Оператор GoTo позволяет передать управление внутрь подпрограммы без использования оператора вызова подпрограммы. В результате пролог не будет выполнен, параметры состояния вычислительного процесса не будут сохранены, что сделает невозможным возврат управления в основной код.

Теорема о структурном программировании

Теорема о структурном программировании.

1965 г., Коррадо Бём, Джузеппе Якопини

Любая программа, заданная в виде блок-схемы, может быть представлена с помощью трёх управляющих структур:

- **последовательность** — *обозначается:* f THEN g,
- **ветвление** — *обозначается:* IF p THEN f ELSE g,
- **цикл** — *обозначается:* WHILE p DO f,

где f, g — блок-схемы с одним входом и одним выходом,
p — условие,
THEN, IF, ELSE, WHILE, DO — ключевые слова.
Пояснение. Формула f THEN g означает следующее: сначала выполняется программа f, затем выполняется программа g.

Принципы структурного программирования

1) Следует отказаться от использования оператора безусловного перехода GoTo.
2) Любая программа строится с использованием трёх базовых управляющих конструкций: последовательность, ветвление, цикл.
   a. Последовательность – это однократное выполнение операторов в том порядке, в котором они записаны в коде программы.
   b. Ветвление – это однократное выполнение одного из двух операторов, в зависимости от истинности или ложности заданного логического условия.
   c. Цикл – это многократное выполнение одного или нескольких операторов до тех пор, пока заданное условие истинно.
3) В программе базовые управляющие конструкции могут быть вложены друг в друга произвольным образом.
4) Повторяющиеся фрагменты кода программы можно оформить в виде подпрограммы (процедуры или функции). Так же возможно оформить в виде подпрограммы логически завершённый фрагмент кода программы, даже если он не повторяется.
5) Каждую логически законченную группу операторов следует оформлять как блок. Блок следует рассматривать как один оператор. Границы блока должны быть строго определены. Блок может быть пустым. Блоки могут быть вложены друг в друга. Например, в алгоритмическом языке C++ блоки выделяются фигурными скобками ({…}).

Принципы структурного программирования, продолжение

6) Программные конструкции, перечисленные в пунктах 4 и 5 должны иметь один вход и один выход. *Замечание:* программирование в стиле спагетти-код допускает наличие программных конструкций, имеющих несколько входов и выходов.
7) Разработка программы ведётся пошагово, методом сверху вниз.

Преимущества структурного программирования:

1) Использование структурного программирования сокращает количество вариантов создания кода программы по одной и той же спецификации
2) Структурное программирование обеспечивает расположение логически связанных операторов ближе друг к другу, а логически не связанных дальше друг от друга. ***Программа становится самодокументированной.***
3) Процесс тестирования и отладки структурированных программ значительно проще по сравнению со спагетти-кодом.

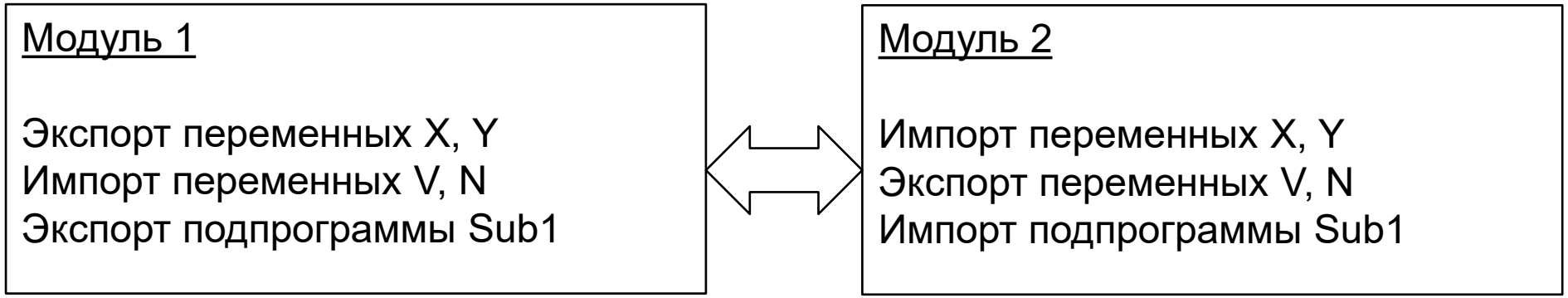
Модульное программирование

Для написания одного модуля должно быть достаточно минимальных знаний о тексте другого.

Давид Парнас, 1972 г.

Интерфейс модуля - набор правил, позволяющих обращаться к программным единицам, содержащимся в модуле с целью передачи им фактических параметров и получения от них результатов вычислений.

Алгоритмический язык Modula, 1975 г., Никлаус Вирт.



Положительные эффекты от использования модулей:

- код модуля можно отлаживать и, при необходимости, модернизировать, не затрагивая код других модулей.
- единожды отладив код модуля, его можно использовать в других программных продуктах без изменения.

Трансляция и компоновка программ

Переход от алгоритмического языка к машинным кодам осуществляется в два этапа:

- трансляция,
- компоновка.

Трансляция:

1) Лексический анализ.
2) Работа с таблицами.
3) Синтаксический анализ.
4) Генерация промежуточного кода.
5) Оптимизация.
6) Генерация объектного кода.

Лексический анализ. Текст кода программы рассматривается, как цепочка символов. Эта цепочка символов подаётся на вход лексического анализатора. Задача лексического анализатора разделить цепочку символов на объекты, называемые лексемами.

Трансляция и компоновка программ

Работа с таблицами. Для ускорения выполнения следующего шага трансляции, лексический анализатор распределяет найденный лексемы по нескольким таблицам:

- зарезервированные слова алгоритмического языка;
- имена переменных, определённых пользователем;
- имена функций, определённых пользователем;
- другое (зависит от алгоритмического языка)

Синтаксический анализ. Цепочка лексем подаётся на вход синтаксического анализатора. Синтаксический анализатор пытается выделить среди последовательности лексем синтаксические конструкции, допустимые в транслируемом алгоритмическом языке. На выходе создаётся дерево синтаксического разбора.

Генерация промежуточного кода. На основании построенного дерева синтаксического разбора формируется промежуточный код программы на языке ассемблера.

Трансляция и компоновка программ

Оптимизация. Делается попытка улучшить код на языке ассемблера с целями повышения его быстродействия и уменьшения объёма. Могут выполняться следующие преобразования:

- удаление бесполезных присваиваний;
- перестановка арифметических операций;
- преобразование и сокращение логических операций:
- вынесение из тела цикла операций, результат которых не изменяется при повторении;
- передача фактических параметров в функцию через регистры процессора, а не через стек.

Генерация объектного кода. После оптимизации код на языке ассемблера переводится в машинные команды. На выходе получается объектный код.

Объектный код выполнить невозможно!

Определение переменной/подпрограммы – размещение в оперативной памяти значения переменной/кода подпрограммы.

Объявление переменной/подпрограммы – «обещание» определить переменную/подпрограмму.

Трансляция и компоновка программ

Таблица адресов объектов включает следующие типы объектов:

- экспортируемый объект. Адрес этого объекта может использоваться в других объектных модулях;
- импортируемый объект. Адрес этого объекта должен быть взят из другого объектного файла, в котором он указан как экспортируемый;
- локальный объект. Адрес этого объекта используется только в текущем объектном файле.

Объектный файл 1	Объектный файл 2
Переменная X1: импорт	Переменная Y1:импорт
Переменная Y1: экспорт: 102378	Переменная Z2:локальная:123897
Подпрограмма P1: импорт	Переменная X1:экспорт:617283
Подпрограмма P2:экспорт: 234987	Подпрограмма P1:экспорт:783492
Переменная Z1:локальная:528791	

Библиотеки подпрограмм

Библиотека подпрограмм - модуль, содержащий несколько подпрограмм, предназначенных для повторного использования.

Типы библиотек подпрограмм:

- *Индивидуальные библиотеки*, используются в одном программном продукте и содержат подпрограммы, без которых программный продукт не может функционировать
- *Библиотеки расширения*, используются для того, чтобы добавить в программный продукт дополнительные функциональные возможности. Библиотека называется внешней.
- *Универсальные библиотеки*, могут использоваться совместно с неограниченным количеством программных продуктов и никак не зависят от программного продукта, в котором используются. Такие библиотеки так же, как библиотеки расширения, называются внешними.

В библиотечных подпрограммах отсутствуют импортируемые объекты.

Статические библиотеки – подключаются на этапе компоновки.
Динамические библиотеки – подключаются на этапе выполнения. Dynamic Link Library (DLL)